

2.3 RNA-seq - Diff. Exp

徐英泽 2024011267

[返回引导页](#)

1. 什么是Multiple test correction? 并解释 q value(很多时候也叫FDR) 和 p value 的差别。

- **多重检验校正 (Multiple test correction)** 是为了控制当同时进行大量统计假设检验时，由于偶然因素导致假阳性概率显著升高的现象而采用的一系列统计调整方法。
- **p值**是在单个假设检验中，当零假设为真时观察到当前或更极端结果的概率，它未考虑多重比较问题；而**q值**（在差异表达分析中常指经FDR校正后的值）是基于对错误发现率（FDR）的控制，表示在所有被判定为显著的检验结果中，预期假阳性所占的最大比例，因此在多重检验场景下，应使用q值（或FDR校正后的p值）而非原始p值来筛选差异特征。

2. 请结合上课时所讲的知识阐述DESeq2和edgeR中如何对数据进行normalization，列出并解释具体的公式。

- **edgeR: TMM counts**。先计算每个样本的原始文库大小 D ，去除在所有样本中表达量极低（或不表达）的基因，减少噪音；其次，从所有样本中选一个**参考样本 r** ，选择标准为该样本的基因表达水平与所有样本的平均水平最接近；接下来，对每个样本 j 与参考样本 r 的每个基因 g 计算**M值**（表达倍数变化的 $\log_2$ 值）与**A值**（表达水平的平均值）；然后，基于负二项分布方差计算每个基因的权重 w_g ；其次，计算 TMM 校正因子，得到缩放因子 C ；最后得到标准化表达量 K 。

$$M_g(j, r) = \log_2 \frac{K_{gj}/D_j}{K_{gr}/D_r}$$

$$A_g(j, r) = \frac{1}{2} (\log_2(K_{gj}/D_j) + \log_2(K_{gr}/D_r))$$

$$w_g(j, r) = \left(\frac{D_j - K_{gj}}{D_j K_{gj}} + \frac{D_r - K_{gr}}{D_r K_{gr}} \right)^{-1}$$

$$\text{TMM}(j, r) = \frac{\sum_{g \in G^*} w_g(j, r) \cdot M_g(j, r)}{\sum_{g \in G^*} w_g(j, r)}$$

$$C_j = 2^{\text{TMM}(j,r)}$$

$$K_{gj}^{(\text{norm})} = \frac{K_{gj}}{C_j}$$

- **DEseq2: RLE**。首先，计算每个基因在各样本中的**几何均值**；其次，用几何均值对每个基因的原始 counts 做缩放；然后，计算样本**j**的标准化因子**C**；最后得到标准化counts，**K**。

$$\left(\prod_{i=1}^N K_{gi} \right)^{1/N}$$

(几何均值)

$$\frac{K_{gj}}{\text{基因 } g \text{ 的几何均值}}$$

$$C_j = \text{median}_g \left(\frac{K_{gj}}{\left(\prod_{i=1}^N K_{gi} \right)^{1/N}} \right)$$

$$K_{gj}^{\text{norm}} = \frac{K_{gj}}{C_j}$$

3. 利用我们以上介绍的方法和数据，分别使用DESeq2和edgeR找出uvr8突变型（uvr8）在光照前后的差异基因，保存为文本文件

```
> setwd("C:/Users/54727/Downloads")
> raw.counts <- read.table("count_exon.txt", sep='\t', header = T, row.names = 1)
# 读取文件。
> uvr8.raw.counts <- raw.counts[,c("UD1_1", "UD1_2", "UD1_3", "UD0_1", "UD0_2", "UD0_3")]
# 取uvr相关的数据。
> uvr8.filtered.counts <- uvr8.raw.counts[rowMeans(uvr8.raw.counts) > 5, ]
# 过滤低表达基因。
> conditions <- factor(c(rep("Control", 3), rep("Treatment", 3)), levels =
c("Control", "Treatment"))
> colData <- data.frame(row.names = colnames(uvr8.filtered.counts), conditions=conditions)
# colData储存样本信息。
```

DESeq2分析代码如下。

```
# DESeq2分析
> library(DESeq2)
> dds <- DESeqDataSetFromMatrix(uvr8.filtered.counts, colData, design = ~conditions)
> dds <- DESeq(dds)
> res <- results(dds)
# 进行分析。
> diff.table <- subset(res, padj < 0.05 & abs(log2FoldChange) > 1)
> write.table(diff.table, "DESeq2.txt", sep='\t', row.names = T, quote = F)
# 过滤后导出结果，过滤标准：padj < 0.05, log2 fold change > 1。
```

edgeR分析代码如下。

```
# edgeR分析
> conditions <- factor(c(rep("Control", 3), rep("Treatment", 3)), levels =
c("Control", "Treatment"))
> design <- model.matrix(~conditions)
# 获取design矩阵。
> library(edgeR)
> y <- DGEList(counts = uvr8.filtered.counts)
# 定义edgeR用于存储基因表达信息的DGEList对象。
> y <- calcNormFactors(y, method="TMM")
# TMM标准化（TMM 实际上是edgeR的默认参数）。
> y <- estimateDisp(y, design = design)
# 估计dispersion。
> fit <- glmFit(y, design = design)
# 拟合广义线性模型。
> lrt <- glmLRT(fit, coef=2)
# 似然比检验。
> diff.table <- topTags(lrt, n = nrow(y))$table
> diff.table.filtered <- diff.table[abs(diff.table$logFC) > 1 & diff.table$FDR < 0.05,]
# 挑选显著变化的基因。
> write.table(diff.table.filtered, file = 'edgeR.txt', sep = "\t", quote = F, row.names = T,
col.names = T)
```

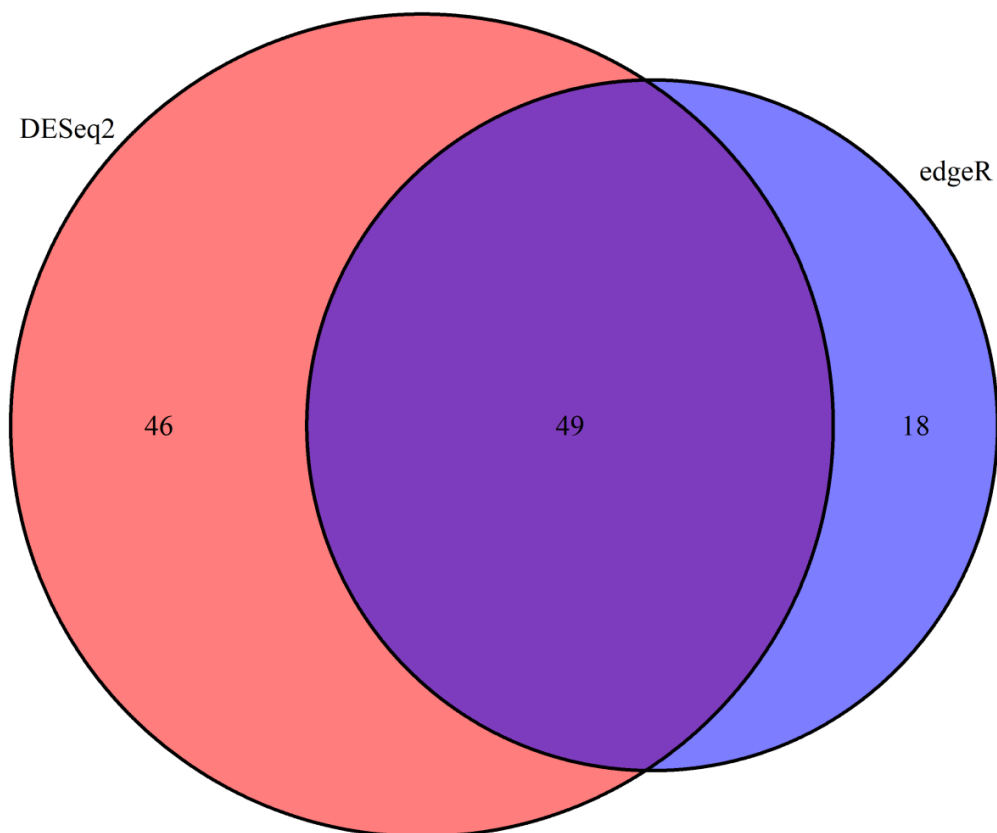
```
# 过滤后导出结果。
```

4. 对于uvr8突变型的差异基因，定义 $|\log_2FC| > 1$ ， $FDR < 0.05$ 的基因为差异表达基因。比较两个软件得到的差异基因有多少是重合的，有多少是不同的，用venn图的形式展示

在上述步骤中，获得的矩阵已经是所需的差异表达基因，重复的代码不在此赘述。

```
> deseq <- read.table("DESeq2.txt", sep = "\t", header = T, row.names = 1)
> edger <- read.table("edger.txt", sep = "\t", header = T, row.names = 1)
# 将差异表达基因名称提取出来。
> library(VennDiagram)
> venn_list <- list(row.names(deseq), row.names(edger))
> names(venn_list) <- c("DESeq2", "edgeR")
> venn.diagram(x=venn_list,
+             fill=c('red', 'blue'),
+             filename = "venn.png")
# 使用VennDiagram进行venn图绘制。
```

结果如下图所示：



此处本应有图片，若未正常显示，则需在压缩包文件夹中进行查看。

5. 对于edgeR找出的FDR<0.05的基因，选出log2FoldChange最大的10个基因和最小的10个基因。计算原始表达量矩阵的log10CPM值并对每个基因进行Z-score处理，使用刚才筛选出来的20个基因绘制热图（heatmap）作为最后结果输出。

```
> library(dplyr)
> edger <- arrange(edger, logFC)
# 按logFC排序。
> target <- rbind(head(edger, 10), tail(edger, 10))
# 选出log2FoldChange最大的10个基因和最小的10个基因。
> min_max.counts <- uvr8.filtered.counts[rownames(target), ]
# 根据排序筛选后的target，在原始表达矩阵中寻找相关基因。
> CPM.minmax <- t(1000000*t(min_max.counts)/colSums(min_max.counts))
> log10.minmax <- log10(CPM.minmax+1)
> z.scores <- (log10.minmax - rowMeans(log10.minmax))/apply(log10.minmax,1,sd)
# 计算原始表达量矩阵的log10CPM值并对每个基因进行Z-score处理。
> library(pheatmap)
> annotation_col = data.frame(Groups = factor(rep(c("Control", "Light Treatment"), c(3,3))))
# 定义热图中每一列的所属分组。
> rownames(annotation_col) = colnames(z.scores)
# 将annotation_col的行名设置为z.scores的列名（样本ID）。
> annotation_row = data.frame(GeneClasses = factor(rep(c("Low LogCPM", "High LogCPM"),
c(10,10))))
# 定义热图中每一行的所属类别（基因表达水平分组）。
> rownames(annotation_row) = colnames(t(z.scores))
# 将annotation_row的行名设置为z.scores的行名（基因ID）。

> library(pheatmap)
> pheatmap(z.scores,
+         cluster_rows = FALSE,
+         cluster_cols = FALSE,
+         annotation_col = annotation_col,
+         annotation_row = annotation_row,
+         color = colorRampPalette(brewer.pal(11, "PuOr"))(50),
+         filename = "heatmap_new.png")
# 绘制热图。
```

绘制热图，如下图所示：

